

Übungsblatt 3

Ausgabe: 11. November 2019

Abgabe: 21. November 2019

Hausübung

Aufgabe 3.1 (Metamodellierung)

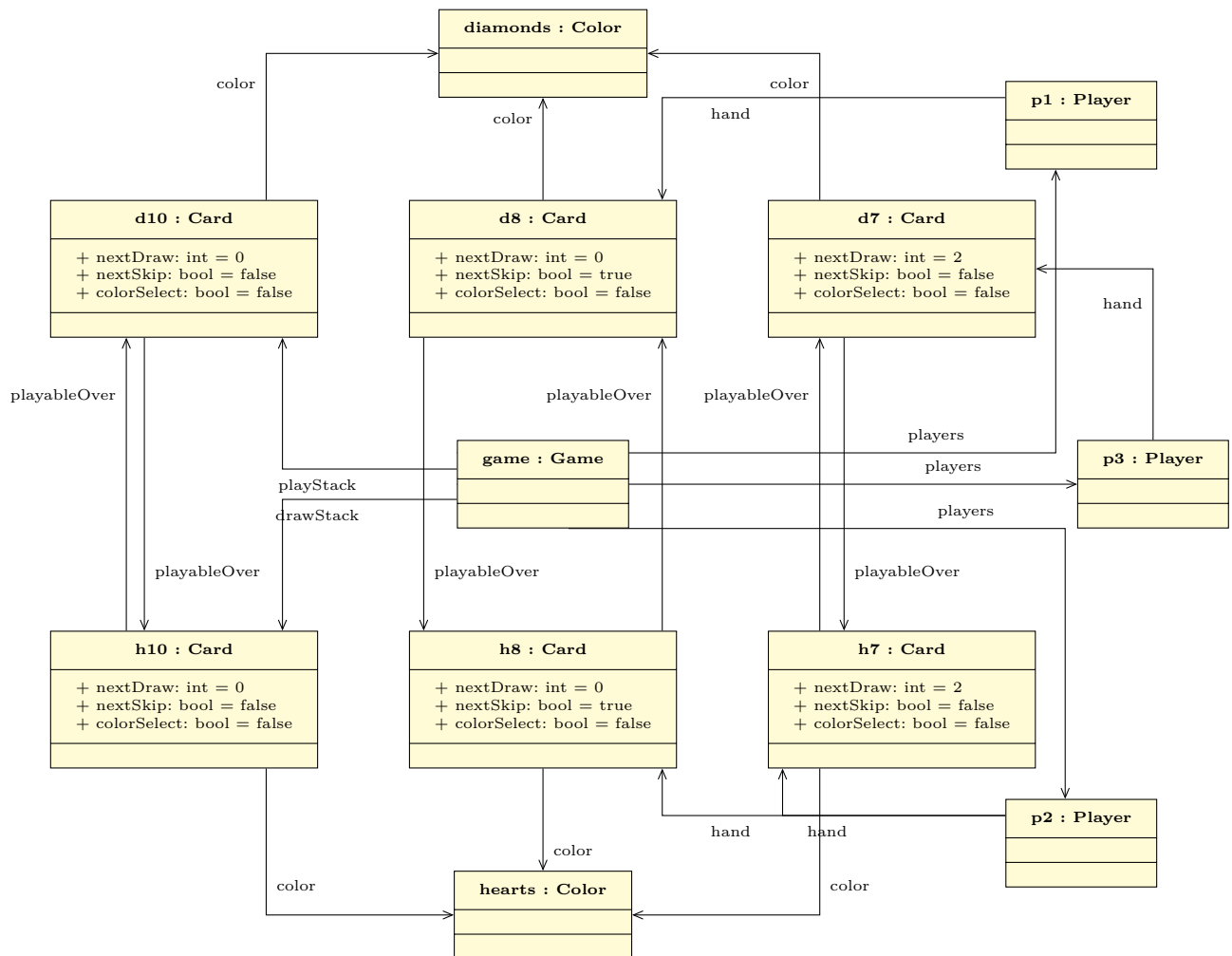
6 P.

Kartenspiele wie Mau-Mau¹ und Uno² werden auf sehr ähnliche Weise gespielt: Reihum legen Spieler Karten von ihrer Hand auf einen Stapel ab. Karten dürfen gelegt werden, wenn sie entweder die gleiche Farbe, oder das gleiche Symbol wie die Karte auf dem Stapel aufweisen. Dürfen Spieler keine Karten ablegen, müssen sie stattdessen eine Karte von einem Stapel ziehen. Gewonnen hat, wer als erstes alle Karten abgelegt hat. Zusätzlich haben einige Karten Sondereffekte (den nächsten Spieler zwei Karten ziehen lassen, den nächsten Spieler aussetzen lassen, eine neue Farbe wählen). Weitere Sondereffekte können Sie vernachlässigen.

Das UML-Diagramm zeigt ein Modell, welches eine (vereinfachte) Partie Mau-Mau¹ mit drei Spielern und sechs Karten (7, 8, 10 in den Farben Herz und Karo) modelliert.

¹[https://de.wikipedia.org/wiki/Mau-Mau_\(Kartenspiel\)](https://de.wikipedia.org/wiki/Mau-Mau_(Kartenspiel))

²[https://de.wikipedia.org/wiki/Uno_\(Kartenspiel\)](https://de.wikipedia.org/wiki/Uno_(Kartenspiel))



- (a) Definieren Sie ein zu dem gegebenen Modell konformes Metamodell in Form eines UML-Klassendiagramms, welches gültige *Stellungen*, d.h. Kombinationen aus Spielern, Handkarten, Ablage- und Nachziehstapel für *beliebige* Decks und Regeln definiert. Zu den Regeln gehört: 6P.

- (i) welche Karten aufeinander gelegt werden dürfen und
- (ii) welche Karte welchen Sondereffekt hat.

Das „Weiterreichen“ von Sondereffekten können Sie vernachlässigen. Es steht Ihnen frei, Ihre Antwort zur besseren Verständlichkeit zu kommentieren. Vergessen Sie nicht, die Multiplizitäten an *beiden* Enden *aller* Relationen anzugeben.

Aufgabe 3.2 (Ausführung von Komponenten)

14 P.

Gegeben seien die Komponenten C_{check} und $C_{abbuchen}$, aus dem auf Blatt 02 beschriebenen UniCard-Bezahlsystem. C_{check} überprüft, ob der Kunde genügend Guthaben auf seinem Konto hat, $C_{abbuchen}$ bucht bei genügend Guthaben den Gesamtpreis der Speisen vom Konto des Kunden ab.

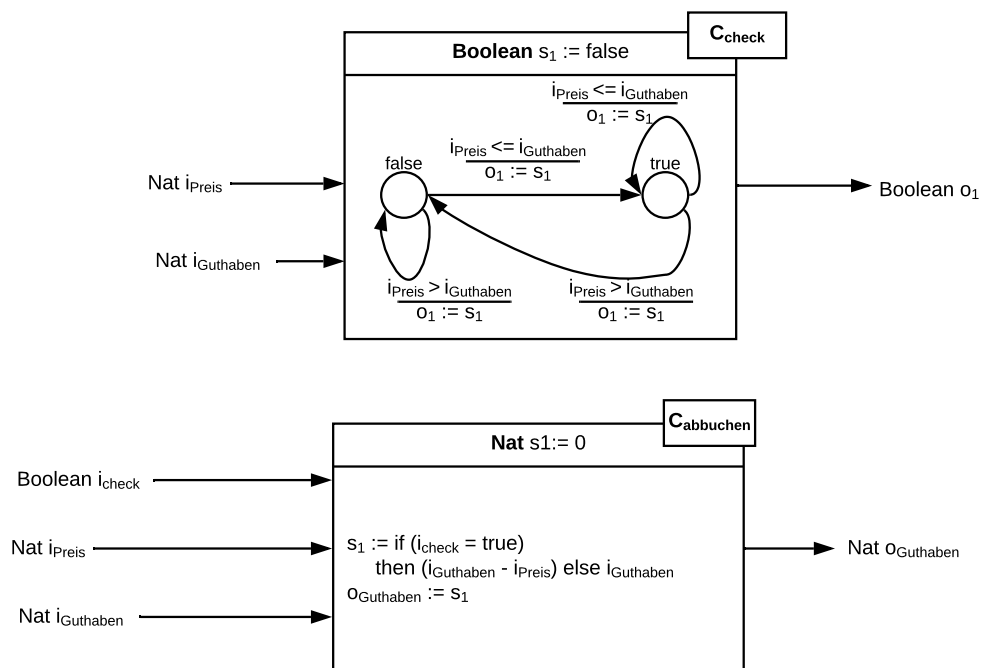


Abbildung 1: Komponenten in konkreter Syntax

- (a) Gehen Sie davon aus, dass die Komponente C_{check} für ein initiales Guthaben von 20€ aufgerufen wird und nacheinander Speisen im Wert von 5€, 8€, 4€ und 6€ bezahlt werden sollen. Ebenfalls können Sie davon ausgehen, dass der Guthabenbetrag nach jeder Ausführung der Komponente C_{check} korrekt um den Preis jeder Speise reduziert wird. 7 P.
- Formulieren Sie für die Komponente C_{check} die abstrakte Syntax der Komponente (vgl. Folie swk-3.2, Folie 37/40/42) (2 Punkte).

- Formulieren Sie für die Komponente C_{check} die Transitionen der Komponente ($\rightarrow$) (vgl Folie **swk-3.2**, Folie 37/40/42) (2 Punkte).
 - Stellen Sie für die Komponente C_{check} die Ausführung (Lauf) in Tabellenform dar, sodass die oben genannte Sequenz von Eingaben bearbeitet wird (vgl Folie **swk-3.2**, Folie 37/40/42) (3 Punkte).
- (b) Gehen Sie davon aus, dass die Komponente $C_{abbuchen}$ für ein initiales Guthaben von 20€ aufgerufen wird und nacheinander Speisen im Wert von 5€, 8€, 4€ und 6€ bezahlt werden sollen. Ebenfalls können Sie davon ausgehen, dass die Überprüfung des Guthabenbetrags vor jeder Ausführung der Komponente $C_{abbuchen}$ korrekt durchgeführt und der Eingang i_{check} somit stets den korrekten Zustand aufweist. 7 P.
- Formulieren Sie für die Komponente $C_{abbuchen}$ die abstrakte Syntax der Komponente (vgl Folie **swk-3.2**, Folie 37/40/42) (2 Punkte).
 - Formulieren Sie für die Komponente $C_{abbuchen}$ die Transitionen der Komponente ($\rightarrow$) (vgl Folie **swk-3.2**, Folie 37/40/42) (2 Punkte).
 - Stellen Sie für die Komponente $C_{abbuchen}$ die Ausführung (Lauf) in Tabellenform dar, sodass die oben genannte Sequenz von Eingaben bearbeitet wird (vgl Folie **swk-3.2**, Folie 37/40/42) (3 Punkte).